

Java Tip 98: Reflect on the Visitor design pattern

Implement visitors in Java, using reflection

By Jeremy Blosser, JavaWorld.com, 07/14/00

Collections are commonly used in object-oriented programming and often raise code-related questions. For example, "How do you perform an operation across a collection of different objects?"

One approach is to iterate through each element in the collection and then do something specific to each element, based on its class. That can get pretty tricky, especially if you don't know what type of objects are in the collection. If you wanted to print out the elements in the collection, you could write a method like this:

```
public void messyPrintCollection(Collection collection) {
    Iterator iterator = collection.iterator()
    while (iterator.hasNext())
        System.out.println(iterator.next().toString())
}
```

That seems simple enough. You just call the `Object.toString()` method and print out the object, right? What if, for example, you have a vector of hashtables? Then things start to get more complicated. You must check the type of object returned from the collection:

```
public void messyPrintCollection(Collection collection) {
    Iterator iterator = collection.iterator()
    while (iterator.hasNext()) {
        Object o = iterator.next();
        if (o instanceof Collection)
            messyPrintCollection((Collection)o);
        else
            System.out.println(o.toString());
    }
}
```

OK, so now you have handled nested collections, but what about other objects that do not return the `String` that you need from them? What if you want to add quotes around `String` objects and add an `f` after `Float` objects? The code gets still more complex:

```
public void messyPrintCollection(Collection collection) {
    Iterator iterator = collection.iterator()
    while (iterator.hasNext()) {
        Object o = iterator.next();
        if (o instanceof Collection)
            messyPrintCollection((Collection)o);
        else if (o instanceof String)
            System.out.println("'" + o.toString() + "'");
        else if (o instanceof Float)
            System.out.println(o.toString() + "f");
        else
            System.out.println(o.toString());
    }
}
```

You can see that things can start to get intricate really fast. You don't want a piece of code with a huge list of if-else statements! How do you avoid that? The Visitor pattern comes to the rescue.

To implement the Visitor pattern, you create a `Visitor` interface for the visitor, and a `Visitable` interface for the collection to be visited. You then have concrete classes that implement the `Visitor` and `Visitable` interfaces. The two interfaces look like this:

```
public interface Visitor
{
    public void visitCollection(Collection collection);
    public void visitString(String string);
    public void visitFloat(Float float);
}
public interface Visitable
{
    public void accept(Visitor visitor);
}
```

For a concrete `String`, you might have:

```
public class VisitableString implements Visitable
{
    private String value;
    public VisitableString(String string) {
        value = string;
    }
    public void accept(Visitor visitor) {
        visitor.visitString(this);
    }
}
```

In the `accept` method, you call the correct visitor method for `this` type:

```
visitor.visitString(this)
```

That lets you implement a concrete `Visitor` as the following:

```
public class PrintVisitor implements Visitor
{
    public void visitCollection(Collection collection) {
        Iterator iterator = collection.iterator()
        while (iterator.hasNext()) {
            Object o = iterator.next();
            if (o instanceof Visitable)
                ((Visitable)o).accept(this);
        }
    }
    public void visitString(String string) {
        System.out.println("'" + string + "'");
    }
    public void visitFloat(Float float) {
        System.out.println(float.toString() + "f");
    }
}
```

By then implementing a `VisitableFloat` class and a `VisitableCollection` class that each call the appropriate visitor methods, you get the same result as the messy if-else `messyPrintCollection` method but with a much cleaner approach. In `visitCollection()`, you call `Visitable.accept(this)`, which in turn calls the correct visitor method. That is called a double-dispatch; the `Visitor` calls a method in the `Visitable` class, which calls back into the `Visitor` class.

Although you've cleaned up an if-else statement by implementing the visitor, you've still introduced a lot of extra code. You've had to wrap your original objects, `String` and `Float`, in objects implementing the `Visitable` interface. Although annoying, that is normally not a problem since the collections you are usually visiting can be made to contain only objects that implement the `Visitable` interface.

Still, it seems like a lot of extra work. Worse, what happens when you add a new `Visitable` type, say `VisitableInteger`? That is one major drawback of the Visitor pattern. If you want to add a new `Visitable` object, you have to change the `Visitor` interface and then implement that method in each of your `Visitor` implementation classes. You could use an abstract base class `Visitor` with default no-op functions instead of an interface. That would be similar to the `Adapter` classes in Java GUIs. The problem with that approach is that you need to use up your single inheritance, which you often want to save for something else, such as extending `StringWriter`. It would also limit you to only be able to visit `Visitable` objects successfully.

Luckily, Java lets you make the Visitor pattern much more flexible so you can add `Visitable` objects at will. How? The answer is by using reflection. With a `ReflectiveVisitor`, you only need one method in your interface:

```
public interface ReflectiveVisitor {
    public void visit(Object o);
}
```

OK, that was easy enough. `Visitable` can stay the same, and I'll get to that in a minute. For now, I'll implement `PrintVisitor` using reflection:

```
public class PrintVisitor implements ReflectiveVisitor {
    public void visitCollection(Collection collection)
    { ... same as above ... }
    public void visitString(String string)
    { ... same as above ... }
    public void visitFloat(Float float)
    { ... same as above ... }
    public void default(Object o)
    {
        System.out.println(o.toString());
    }
    public void visit(Object o) {
        // Class.getName() returns package information as well.
        // This strips off the package information giving us
        // just the class name
        String methodName = o.getClass().getName();
        methodName = "visit"+
            methodName.substring(methodName.lastIndexOf('.')+1);
        // Now we try to invoke the method visit<methodName>
        try {
            // Get the method visitFoo(Foo foo)
            Method m = getClass().getMethod(methodName,
                new Class[] { o.getClass() });
            // Try to invoke visitFoo(Foo foo)
            m.invoke(this, new Object[] { o });
        }
```

```

    } catch (NoSuchMethodException e) {
        // No method, so do the default implementation
        default(o);
    }
}
}

```

Now you don't need the `Visitable` wrapper class. You can just call `visit()`, and it will dispatch to the correct method. One nice aspect is that `visit()` can dispatch however it sees fit. It doesn't have to use reflection -- it can use a totally different mechanism.

With the new `PrintVisitor`, you have methods for `Collections`, `Strings`, and `Floats`, but then you catch all the unhandled types in the catch statement. You'll expand upon the `visit()` method so that you can try all the superclasses as well. First, you'll add a new method called `getMethod(Class c)` that will return the method to invoke, which looks for a matching method for all the superclasses of `Class c` and then all the interfaces for `Class c`.

```

protected Method getMethod(Class c) {
    Class newc = c;
    Method m = null;
    // Try the superclasses
    while (m == null && newc != Object.class) {
        String method = newc.getName();
        method = "visit" + method.substring(method.lastIndexOf('.') + 1);
        try {
            m = getClass().getMethod(method, new Class[] {newc});
        } catch (NoSuchMethodException e) {
            newc = newc.getSuperclass();
        }
    }
    // Try the interfaces. If necessary, you
    // can sort them first to define 'visitable' interface wins
    // in case an object implements more than one.
    if (newc == Object.class) {
        Class[] interfaces = c.getInterfaces();
        for (int i = 0; i < interfaces.length; i++) {
            String method = interfaces[i].getName();
            method = "visit" + method.substring(method.lastIndexOf('.') + 1);
            try {
                m = getClass().getMethod(method, new Class[] {interfaces[i]});
            } catch (NoSuchMethodException e) {}
        }
    }
    if (m == null) {
        try {
            m = thisclass.getMethod("visitObject", new Class[] {Object.class});
        } catch (Exception e) {
            // Can't happen
        }
    }
    return m;
}

```

It looks complicated, but it really isn't. Basically, you just look for methods based on the name of the class you have passed in. If you don't find one, you try its superclasses. Then if you don't find any of those, you try any interfaces. Lastly, you can just try `visitObject()` as a default.

Note that for the sake of those familiar with the traditional Visitor pattern, I've followed the same naming convention for the method names. However, as some of you may have noticed, it would be more efficient

to name all the methods "visit" and let the parameter type be the differentiator. If you do that, however, make sure you change the main `visit(Object o)` method name to something like `dispatch(Object o)`. Otherwise, you won't have a default method to fall back on, and you would need to cast to `Object` whenever you call `visit(Object o)` to assure the correct method calling pattern was followed.

Now, you modify the `visit()` method to take advantage of `getMethod()`:

```
public void visit(Object object) {
    try {
        Method method = getMethod(getClass(), object.getClass());
        method.invoke(this, new Object[] {object});
    } catch (Exception e) { }
}
```

Now, your visitor object is much more powerful. You can pass in any arbitrary object and have some method that uses it. Plus, you gain the added benefit of having a default method `visitObject(Object o)` that can catch anything you don't specify. With a little more work, you can even add a method for `visitNull()`.

I've kept the `Visitable` interface in there for a reason. Another side benefit of the traditional Visitor pattern is that it allows the `Visitable` objects to control navigation of the object structure. For example, if you had a `TreeNode` object that implemented `Visitable`, you could have an `accept()` method that traverses to its left and right nodes:

```
public void accept(Visitor visitor) {
    visitor.visitTreeNode(this);
    visitor.visitTreeNode(leftsubtree);
    visitor.visitTreeNode(rightsubtree);
}
```

So, with just one more modification to the `Visitor` class, you can allow for `Visitable`-controlled navigation:

```
public void visit(Object object) throws Exception
{
    Method method = getMethod(getClass(), object.getClass());
    method.invoke(this, new Object[] {object});
    if (object instanceof Visitable)
    {
        callAccept((Visitable) object);
    }
}
public void callAccept(Visitable visitable) {
    visitable.accept(this);
}
```

If you've implemented a `Visitable` object structure, you can keep the `callAccept()` method as is and use `Visitable`-controlled navigation. If you want to navigate the structure within the visitor, you just override the `callAccept()` method to do nothing.

The power of the Visitor pattern comes into play when using several different visitors across the same collection of objects. For example, I have an interpreter, an infix writer, a postfix writer, an XML writer, and a SQL writer working across the same collection of objects. I could easily write a prefix writer or a

SOAP writer for the same collection of objects. In addition, those writers can gracefully work with objects they don't know about or, if I choose, they can throw an exception.

About the author

Jeremy Blosser has been programming in Java for five years, during which he has worked for various software companies. He now works for a startup company, Software Instruments. You can visit Jeremy's Website at <http://www.blosser.org>.

Conclusion

By using Java reflection, you can enhance the Visitor design pattern to provide a powerful way to operate on object structures, giving the flexibility to add new `Visitable` types as needed. I hope you are able to use that pattern somewhere in your coding travels.

All contents copyright 1995-2013 Java World, Inc. <http://www.javaworld.com>